



DEPARTMENT OF COMPUTER ENGINEERING (DCE)
SOC DESIGN
MMH: SOCD334264

Group: Ae VI MẠCH k24

Name: Lê Quốc Khánh, Nguyễn Mai Trung Dũng, Lê Minh Hải, Đặng Quang Được, Lã Phan Ân Kỳ, Nguyễn Hữu Đức

Câu 1: Phân tích ưu điểm và nhược điểm của ASIC-based SoC so với FPGA-based SoC.

Trả lời

ASIC-based SoC là hệ thống được triển khai thành chip chuyên dụng, trong đó các khối logic và liên kết vật lý được xác định khi chế tạo. FPGA-based SoC sử dụng phần logic có thể cấu hình lại; CPU có thể là hard-core tích hợp sẵn hoặc soft-core được tổng hợp trên FPGA. Vì vậy, sự khác biệt chủ yếu nằm ở cách hiện thực phần cứng và mức độ linh hoạt của hệ thống.



ASIC SoC vs. FPGA SoC

	ASIC SoC (Fixed-Function)	FPGA SoC (Reconfigurable)
Hardware	Fixed, immutable silicon blocks.	CPU subsystem combined with a blank canvas of Programmable Logic (PL).
Flexibility	Low (Cannot be altered post-fabrication).	Extreme (Logic and accelerators reconfigured instantly via bitstream).
Performance/Power	Ultimate optimization; highest speed, lowest power.	Lower than ASIC due to routing overhead in the logic fabric.
Economics	Massive upfront NRE and mask costs; cheap at millions of units.	Zero mask costs; ideal for fast prototyping and low-volume production.
Examples	Broadcom BCM2711, Apple A-Series, Snapdragon.	Xilinx Zynq UltraScale+, Intel Cyclone V.



Ưu điểm của ASIC-based SoC

Về hiệu năng, ASIC cho phép tối ưu trực tiếp đường dữ liệu, độ rộng toán hạng, cấu trúc bộ nhớ và liên kết theo ứng dụng. So với việc ánh xạ cùng chức năng vào LUT và mạng định tuyến lập trình được của FPGA, ASIC thường có đường truyền ngắn hơn, ít phần tử trung gian hơn và dễ đạt hiệu năng cao hơn trong cùng điều kiện so sánh.

Về công suất và diện tích, ASIC không cần phần lớn tài nguyên phục vụ cấu hình như bộ nhớ cấu hình và các bộ chọn đường tín hiệu của FPGA. Điện dung chuyển mạch và số transistor phụ trợ thường thấp hơn; đồng thời có thể áp dụng clock gating, power gating và tối ưu thư viện cell. Do đó ASIC thường có hiệu quả năng lượng và mật độ tích hợp tốt hơn cho cùng tác vụ.

Về chi phí sản phẩm, ASIC có thể đạt chi phí trên mỗi đơn vị thấp khi sản xuất số lượng lớn, vì chi phí thiết kế và chế tạo ban đầu được phân bổ cho nhiều chip. ASIC cũng thuận lợi cho tích hợp CPU, bộ nhớ, ngoại vi, khối analog và bộ tăng tốc chuyên dụng nếu công nghệ cùng các IP tương ứng hỗ trợ.

Nhược điểm của ASIC-based SoC

Chi phí không lặp lại (NRE) cao: phải đầu tư thiết kế, công cụ EDA, IP, kiểm chứng, mask và chế tạo thử. Thời gian đưa sản phẩm ra thị trường thường dài hơn do phải hoàn thành sign-off, tape-out, chế tạo và kiểm thử silicon.

Khả năng sửa đổi phần cứng sau chế tạo thấp. Có thể cập nhật phần mềm chạy trên CPU, nhưng lỗi trong logic cố định thường cần phương án khắc phục bằng phần mềm hoặc một lần chế tạo lại. Ngược lại, FPGA có thể sửa phần logic bằng bitstream mới trong giới hạn tài nguyên và phần cứng có sẵn.

ASIC chịu rủi ro lớn hơn khi yêu cầu chưa ổn định hoặc sản lượng thấp: một lần sửa silicon có thể tốn nhiều thời gian và chi phí. FPGA phù hợp hơn cho thử nghiệm, nghiên cứu và sản phẩm cần thay đổi thuật toán hoặc giao thức thường xuyên.

Tiêu chí	ASIC-based SoC	FPGA-based SoC
Hiệu năng, năng lượng	Thường tối ưu hơn cho tác vụ cố định.	Chịu chi phí của LUT và định tuyến lập trình được.
Chi phí ban đầu	Cao; có NRE và chi phí chế tạo.	Thấp hơn; dùng chip FPGA có sẵn.
Chi phí khi sản lượng lớn	Có thể thấp hơn sau điểm hòa vốn.	Giá chip thường cao hơn cho cùng chức năng.
Thời gian phát triển	Dài; có chu kỳ chế tạo silicon.	Ngắn hơn cho tạo mẫu và cập nhật.
Thay đổi phần cứng	Khó sau chế tạo.	Có thể nạp lại cấu hình.

Có thể mô hình hóa tổng chi phí là $C = NRE + N \times c$, với N là sản lượng và c là chi phí mỗi chip. ASIC có lợi về tổng chi phí khi phần tiết kiệm trên mỗi chip đủ bù NRE lớn hơn. Không thể kết luận ASIC luôn rẻ hơn hoặc nhanh hơn mọi FPGA nếu chưa xác định công nghệ, cấu hình và ứng dụng.

Câu 2: Giải thích kiến trúc PS và PL trong Zynq, đồng thời nêu vai trò của giao tiếp AXI.

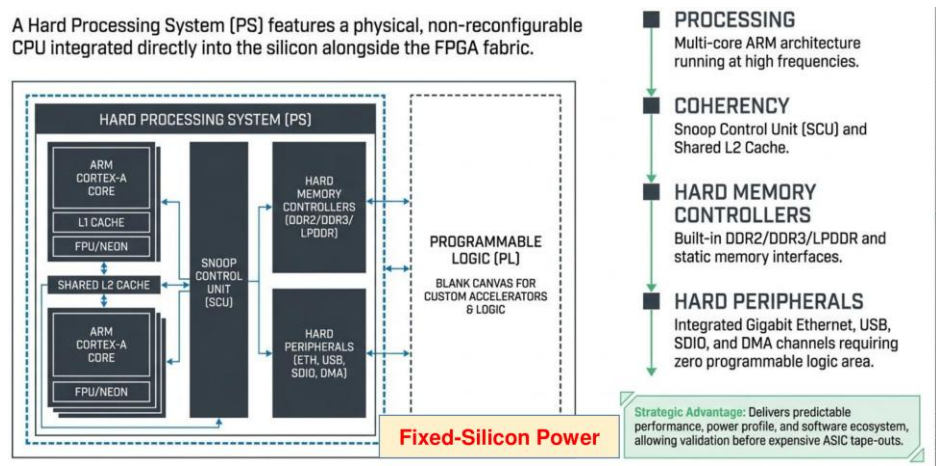
Trả lời

Zynq kết hợp hệ thống xử lý bằng phần mềm và logic cấu hình được trên cùng chip. Trong đó, Zynq-7000 được dùng làm ví dụ; không áp dụng cấu hình CPU này cho mọi thế hệ Zynq.

PS và PL

PS (Processing System) là phần xử lý đã được chế tạo cố định, gồm ARM Cortex-A9, cache, bộ nhớ trên chip, bộ điều khiển DDR, bộ điều khiển ngắt và các ngoại vi như UART, SPI, I²C, Ethernet. Tùy thiết bị, CPU có một hoặc hai lõi. PS chạy chương trình bare-metal, RTOS hoặc Linux, phù hợp cho quản lý hệ thống, giao thức và điều khiển luồng công việc. PS là phần cứng cố định nhưng hành vi ứng dụng thay đổi được bằng phần mềm.

PL (Programmable Logic) gồm LUT, flip-flop, BRAM, DSP và tài nguyên định tuyến, clock, I/O có thể cấu hình. Người thiết kế dùng RTL hoặc IP sinh bởi HLS để tạo mạch song song, bộ tăng tốc xử lý ảnh, bộ lọc tín hiệu hoặc giao tiếp riêng. Cấu hình PL được nạp bằng bitstream.



Vai trò của AXI

AXI (Advanced eXtensible Interface) thuộc họ AMBA, quy định cách các khối trao đổi địa chỉ, dữ liệu và phản hồi. Với giao tiếp memory-mapped, PS có thể truy cập thanh ghi của IP trong PL; bộ tăng tốc hoặc DMA trong PL có thể đọc, ghi bộ nhớ phía PS. Cơ chế VALID/READY cho phép bên nhận điều tiết luồng dữ liệu. AXI memory-mapped tách các kênh địa chỉ đọc, dữ liệu đọc, địa chỉ ghi, dữ liệu ghi và phản hồi ghi.

Dạng giao tiếp	Vai trò điển hình
AXI4-Lite	Truy cập thanh ghi điều khiển, trạng thái và tham số; giao dịch đơn giản, không truyền burst.
AXI4 memory-mapped	Truyền dữ liệu có địa chỉ, hỗ trợ burst; thường dùng khi trao đổi khối dữ liệu với bộ nhớ.
AXI4-Stream	Truyền luồng dữ liệu giữa các khối xử lý, không mang địa chỉ bộ nhớ trên từng giao dịch; thường kết hợp DMA.

Trên Zynq-7000, M_AXI_GP cho PS khởi tạo truy cập tới PL; S_AXI_GP cho PL khởi tạo truy cập tới PS. Các cổng S_AXI_HP phục vụ luồng dữ liệu hiệu năng cao tới DDR/OCM; S_AXI_ACP hỗ

trợ truy

cập có tính nhất quán cache. Cổng AXI tại biên PS của thể hệ này dựa trên AXI3; việc nối IP AXI4 ở PL có thể cần interconnect hoặc bộ chuyển đổi giao thức phù hợp. Không nên gọi tất cả cổng PS–PL là AXI4.

Ví dụ phối hợp xử lý

Với hệ thống lọc ảnh, PS chuẩn bị vùng nhớ, ghi địa chỉ và kích thước ảnh vào thanh ghi điều khiển. DMA chuyển ảnh từ DDR sang bộ lọc PL qua AXI4-Stream, rồi đưa kết quả về DDR. PL báo hoàn tất bằng ngắt; PS tiếp tục xử lý hoặc hiển thị. Cần quản lý cache đúng cách nếu dùng đường truyền không nhất quán cache và đo cả thời gian truyền dữ liệu khi đánh giá tăng tốc.

Câu 3: Tại sao thiết kế reuse và IP integration lại quan trọng trong SoC design hiện đại?

Trả lời

Trả lời

Trong thiết kế SoC hiện đại, thiết kế reuse (tái sử dụng) và IP integration (tích hợp các khối IP) rất quan trọng vì độ phức tạp của chip ngày càng tăng, việc thiết kế toàn bộ hệ thống từ đầu là không khả thi về thời gian, chi phí và nhân lực.

SoC hiện đại phát triển theo hướng IP-based design và platform-based design nhằm tái sử dụng các khối thiết kế có sẵn.

1. Giảm thời gian thiết kế

- Một SoC có thể chứa nhiều thành phần:

- CPU core.
- Memory.
- Bus interconnect.
- Peripheral.
- Accelerator.
- Analog IP.
- Software/Firmware.

- Nếu thiết kế từng khối từ đầu sẽ mất rất nhiều thời gian.

- Việc sử dụng lại các IP đã được thiết kế và kiểm chứng giúp rút ngắn quá trình phát triển.

2. Giảm chi phí thiết kế

Thiết kế SoC yêu cầu chi phí lớn cho:

- Thiết kế phần cứng.
- Verification.
- Fabrication.
- Testing.

Khi sử dụng IP có sẵn, nhóm thiết kế chỉ cần tích hợp và kiểm tra sự tương thích thay vì phát triển lại toàn bộ.

3. Tăng độ tin cậy của hệ thống

Các IP core thường đã được:

- mô phỏng.
- kiểm thử.
- đánh giá hiệu năng trước khi đưa vào SoC.

Vì vậy giảm nguy cơ lỗi thiết kế khi tích hợp hệ thống lớn.

4. Cho phép phát triển SoC có độ phức tạp cao

Một SoC hiện đại không chỉ gồm CPU mà còn có nhiều khối chuyên dụng như:

- GPU/NPU/DSP accelerator.
- Memory controller.
- Communication interface.
- Peripheral controller.

Chap 2 mô tả SoC bao gồm nhiều khối chức năng được kết nối thông qua các hệ thống interconnection như AXI, AHB, APB để tạo thành một hệ thống hoàn chỉnh.

Nhờ IP reuse, nhà thiết kế có thể xây dựng các hệ thống phức tạp bằng cách ghép các module thay vì thiết kế lại từng phần.

5. Tăng khả năng tùy biến và mở rộng

Các loại IP có mức độ linh hoạt khác nhau:

- **Hard IP:** tối ưu hiệu năng nhưng ít linh hoạt.
- **Soft IP:** mô tả bằng HDL (Verilog/VHDL), dễ thay đổi và tái sử dụng.
- **Firm IP:** cân bằng giữa hiệu năng của Hard IP và tính linh hoạt của Soft IP.

Nhờ đó, nhà thiết kế có thể lựa chọn IP phù hợp với từng ứng dụng như IoT, AI accelerator hoặc hệ thống nhúng.

Kết luận:

Reuse và IP integration quan trọng trong SoC design hiện đại vì giúp giảm thời gian phát triển, giảm chi phí, tăng độ tin cậy và cho phép xây dựng các hệ thống SoC phức tạp bằng cách kết hợp các khối IP đã được thiết kế và kiểm chứng. Xu hướng thiết kế SoC hiện nay dựa trên IP-based design và platform-based design nhằm đáp ứng yêu cầu hiệu năng cao, mật độ tích hợp lớn và thời gian đưa sản phẩm ra thị trường nhanh hơn.

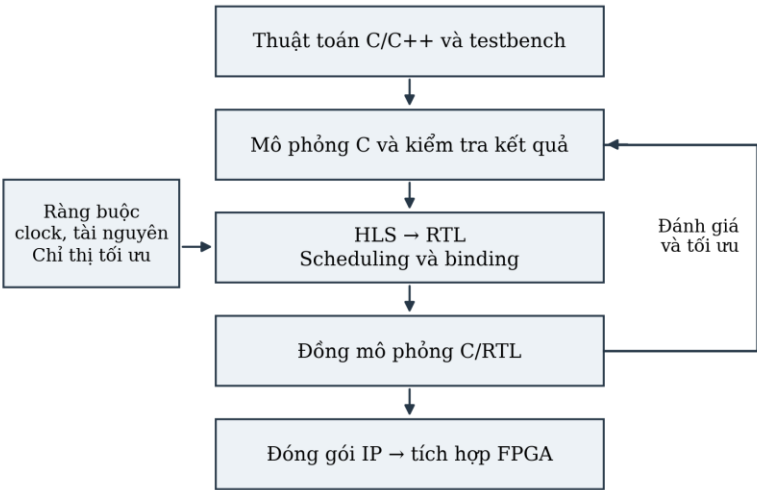
Câu 4: Phân tích vai trò của High-Level Synthesis (HLS) trong việc tăng tốc quá trình phát triển hardware accelerator trên FPGA.

Trả lời

HLS chuyển mô tả thuật toán ở mức C/C++ thành cấu trúc RTL thực hiện chức năng tương ứng. Công cụ phân lịch phép toán theo chu kỳ clock (scheduling), gán phép toán cho tài nguyên phần cứng (binding), tạo đường dữ liệu và điều khiển. Vì vậy, HLS rút ngắn thời gian từ thuật toán đến một IP phần cứng có thể tích hợp.

Rút ngắn thiết kế và kiểm chứng

Người thiết kế có thể kiểm tra thuật toán bằng C simulation trước khi sinh RTL, sau đó dùng C/RTL co-simulation để đối chiếu kết quả. Việc thay đổi thuật toán ở mức cao thường nhanh hơn sửa đồng thời datapath, FSM và testbench RTL. Tuy nhiên, vẫn cần xác định kiến trúc tổng thể, kiểu dữ liệu, bộ nhớ và ràng buộc clock.



Sau tích hợp: synthesis, place & route, bitstream, thử trên board

Hình 2. Quy trình phát triển và tối ưu bộ tăng tốc bằng HLS.

Khám phá các phương án phần cứng

Từ một thuật toán, người thiết kế có thể thay đổi chỉ thị và ràng buộc để tạo nhiều hiện thực, rồi so sánh độ trễ, thông lượng và tài nguyên. Các kỹ thuật dưới đây giải thích những lựa chọn thường gặp; mức cải thiện phụ thuộc dữ liệu, phụ thuộc vòng lặp và băng thông bộ nhớ.

Kỹ thuật	Tác dụng và đánh đổi
Pipeline	Cho các vòng lặp hoặc tác vụ chồng lấp theo thời gian. Giảm khoảng khởi động II nếu các phụ thuộc và bộ nhớ cho phép; có thể tăng số thanh ghi.
Unroll	Nhân bản phần cứng cho nhiều lượt lặp chạy song song. Tăng nhu cầu LUT/DSP và số cổng truy cập bộ nhớ.
Dataflow	Cho các công đoạn đọc, tính toán và ghi chạy đồng thời qua FIFO; phải kiểm tra tốc độ các công đoạn và tránh deadlock.

Kỹ thuật	Tác dụng và đánh đổi
Array partition	Chia mảng thành các bank hoặc thanh ghi để tăng truy cập song song; có thể tốn thêm tài nguyên.
Độ rộng dữ liệu	Dùng số nguyên hoặc fixed-point đủ chính xác giúp giảm tài nguyên; cần đánh giá tràn số và sai số lượng tử.

Ví dụ và giới hạn

Xét bộ lọc FIR: mỗi mẫu đầu ra là tổng các tích giữa hệ số và mẫu đầu vào. HLS có thể tạo cấu trúc dùng chung bộ nhân để tiết kiệm tài nguyên, hoặc unroll để dùng nhiều bộ nhân song song, kết hợp pipeline để tăng thông lượng. Đây là đánh đổi giữa diện tích và tốc độ, không phải cứ thêm chỉ thị là đạt hiệu năng mong muốn.

Nếu một pipeline xử lý một mẫu mỗi lần khởi động, không bị stall và có II chu kỳ giữa hai lần khởi động, thông lượng lý tưởng là f_{clk}/II mẫu mỗi giây. $II = 1$ nghĩa là có thể nhận một mẫu mới mỗi chu kỳ sau khi pipeline hoạt động ổn định; không có nghĩa độ trễ của một mẫu chỉ bằng một chu kỳ.

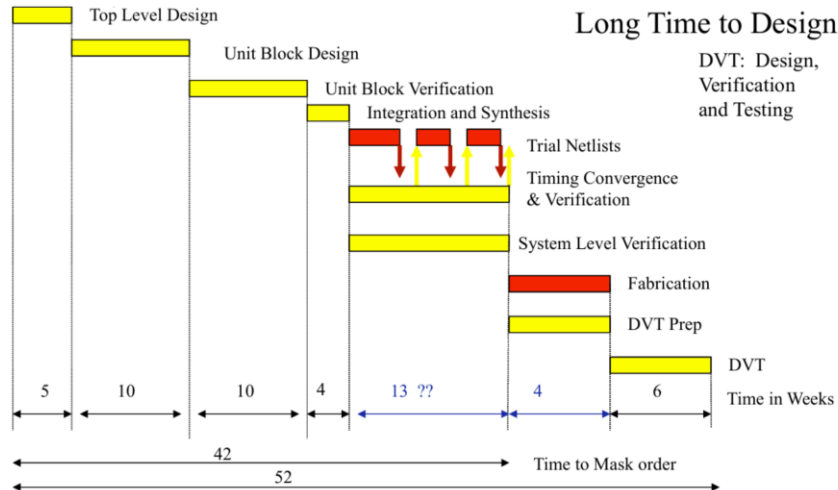
HLS còn hỗ trợ tạo giao tiếp để đóng gói IP, giúp tích hợp bộ tăng tốc với PS hoặc DMA. Tuy nhiên, thời gian toàn hệ thống gồm điều khiển, truyền dữ liệu và tính toán; một accelerator nhanh vẫn có thể không mang lại lợi ích nếu chi phí di chuyển dữ liệu lớn.

RTL sinh ra vẫn phải qua tổng hợp logic, place-and-route, kiểm tra timing và chạy thử trên board. Báo cáo HLS là cơ sở ước lượng, không thay thế kết quả triển khai. HLS đặc biệt hữu ích cho thuật toán xử lý tín hiệu, ảnh và học máy; các mạch điều khiển rất đặc thù có thể vẫn cần RTL viết tay.

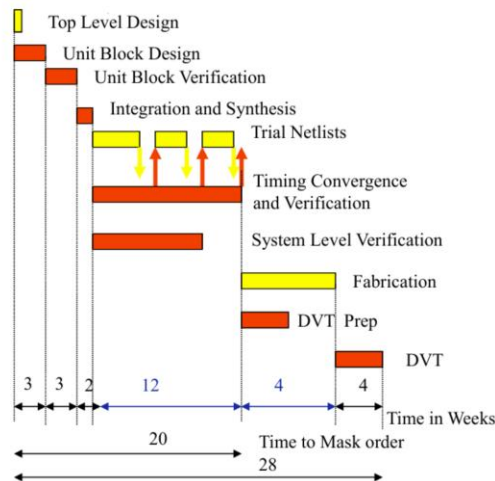
Câu 5: So sánh và phân tích SoC Design Flow vs. ASIC Design Flow.

Trả lời

SoC Design Flow và ASIC Design Flow không phải hai quy trình loại trừ nhau. SoC nói đến phạm vi hệ thống tích hợp, còn ASIC nói đến cách hiện thực chip chuyên dụng. Một SoC có thể triển khai bằng ASIC hoặc FPGA. Với ASIC-based SoC, quy trình ASIC là phần hiện thực phần cứng nằm trong quy trình phát triển hệ thống.



Hình 3a. ASIC Design Flow



- Due to Chip Complexity and lower IC area, it is difficult to reduce Placement, Layout and Fabrication steps time.
- There is need to reduce the time of other steps before Placement, Layout and Fabrication steps.
- One should consider Chip Layout issues up-front.

Hình 3b. SoC Design Flow

Phân tích trực tiếp hai hình trong bài giảng

Hình 3a minh họa cách triển khai truyền thống, trong đó thiết kế top-level, thiết kế từng khối, kiểm chứng và tích hợp chiếm nhiều thời gian. Hình 3b nhấn mạnh việc rút ngắn các giai đoạn đầu nhờ tổ chức thiết kế ở mức hệ thống, tái sử dụng IP và thực hiện các hoạt động phù hợp song song. Phải quan tâm sớm đến layout và timing để hạn chế vòng lặp sửa netlist ở giai đoạn sau.

Thời gian tới đặt mask giảm từ 42 xuống 20 tuần và tổng thời gian từ 52 xuống 28 tuần. Đây là ví dụ giảng dạy, không phải thời gian cố định của mọi dự án. Placement, layout và fabrication khó rút ngắn đáng kể, nên cần giảm thời gian các bước trước đó và xem xét vấn đề vật lý ngay từ đầu.

Quy trình thiết kế SoC

Bước 1: xác định chức năng, hiệu năng, công suất, chi phí, thời gian đáp ứng và các giao tiếp. Bước 2: mô hình hóa, lựa chọn CPU, bộ nhớ, interconnect và IP; phân chia tác vụ cho phần cứng và phần mềm dựa trên đặc tính tính toán cùng chi phí truyền dữ liệu.

Bước 3: phát triển phần cứng, phần mềm và giao tiếp song song. Phần cứng có thể được viết bằng RTL hoặc sinh từ HLS; phần mềm gồm boot code, driver, RTOS/OS và ứng dụng. Bước 4: tích hợp và đồng kiểm chứng HW/SW, đánh giá chức năng, ngắt, bản đồ bộ nhớ và hiệu năng; nếu không đạt thì điều chỉnh phân chia hoặc kiến trúc.

Bước 5: triển khai theo nền tảng. Với FPGA, tạo bitstream và nạp phần mềm để thử trên board. Với ASIC, thực hiện tổng hợp, thiết kế vật lý và chế tạo chip. Bước 6: kiểm thử toàn hệ thống và hoàn thiện phần mềm trên phần cứng thực tế.

Quy trình ASIC và ví dụ trong tài liệu Synopsys

Ở mức tổng quát, ASIC flow bắt đầu từ đặc tả, kiến trúc, RTL và kiểm chứng chức năng. RTL được tổng hợp thành gate-level netlist theo thư viện công nghệ và ràng buộc thiết kế. DFT, scan và ATPG được lập kế hoạch/tích hợp để hỗ trợ kiểm thử sản xuất; đây là nội dung của một flow ASIC đầy đủ, không phải tất cả đều được triển khai trong bộ script FIFO được cung cấp.

Trong , Design Compiler thực hiện tổng hợp logic. IC Compiler II tiếp nhận netlist và thư viện, thiết lập các corner/mode (MCM), xác định floorplan, bố trí pad, xây dựng mạng nguồn, placement, clock tree synthesis và routing. Sau routing cần tối ưu và kiểm tra các vi phạm còn lại.

Floorplanning xác định die/core và tổ chức không gian; placement đặt cell; CTS phân phối clock với skew và tải phù hợp; routing tạo liên kết kim loại. Các bước này phải phối hợp để đáp ứng công suất, hiệu năng và diện tích (PPA), thay vì tối ưu riêng từng bước.

Cuối flow cần kiểm tra vật lý DRC/LVS, trích xuất ký sinh, STA sign-off, phân tích IR-drop và electromigration, rồi xuất GDSII cho tape-out. Sau chế tạo còn có đóng gói, kiểm thử và đánh giá silicon. Mục 12.4 của nêu rõ bộ script kết thúc ở route_done; các bước sign-off và GDSII là phần tiếp theo ngoài bộ script, không được xem là đã hoàn thành chỉ nhờ route_opt.

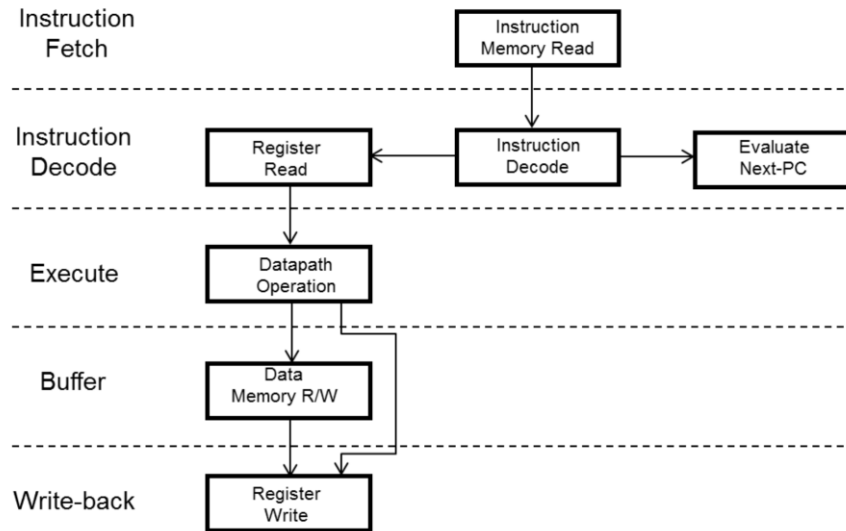
Tiêu chí	SoC Design Flow	ASIC Design Flow
Phạm vi chính	Toàn hệ thống: CPU, IP, bộ nhớ, phần mềm và giao tiếp.	Hiện thực phần cứng thành chip chuyên dụng.
Quyết định nổi bật	Phân chia HW/SW, chọn IP, tổ chức hệ thống.	Chọn công nghệ, ràng buộc, DFT và tối ưu vật lý.
Kiểm chứng	Chức năng hệ thống, HW/SW và hiệu năng ứng dụng.	RTL/netlist, timing, vật lý và khả năng kiểm thử.
Sản phẩm đầu ra	Hệ thống cùng phần mềm; có thể trên FPGA hoặc ASIC.	Dữ liệu chế tạo như GDSII và chip sau sản xuất.

Câu 6: Giải thích sơ đồ khối của kiến trúc RISC-V, trình bày các kỹ thuật tối ưu áp dụng trên kiến trúc RISC-V, phân tích các ứng dụng có sử dụng kiến trúc RISC-V.

Trả lời

Khái niệm và sơ đồ khối

RISC-V là kiến trúc tập lệnh mở (ISA), quy định giao diện mà phần mềm nhìn thấy, gồm lệnh, thanh ghi và hành vi thực thi. ISA không bắt buộc một cấu trúc pipeline duy nhất: có thể hiện thực bằng lõi nhiều chu kỳ, pipeline ngắn hoặc pipeline sâu hơn. Hình 4 là pipeline RISC tổng quát của bài giảng, dùng để giải thích một cách hiện thực RISC-V năm tầng; không phải sơ đồ riêng của một lõi RISC-V cụ thể. Trên hình, tầng “Buffer” thực hiện Data Memory R/W, tương ứng tầng MEM trong phần giải thích dưới đây.



Hình 4. Pipeline RISC tổng quát

Tầng IF (Instruction Fetch) dùng PC để lấy lệnh từ bộ nhớ lệnh hoặc I-cache. Mạch chọn PC xác định địa chỉ lệnh tiếp theo, thông thường $PC + 4$ với lệnh RV32I dài 32 bit; khi hỗ trợ lệnh nén 16 bit, bước tăng còn phụ thuộc độ dài lệnh. Lệnh nhánh, nhảy hoặc ngoại lệ có thể chuyển PC tới địa chỉ khác. Việc tính PC đích có thể được bố trí ở những tầng khác nhau tùy vi kiến trúc; hình gốc đặt khối Evaluate Next-PC cạnh tầng giải mã.

Tầng ID (Instruction Decode) giải mã opcode, nhận dạng các thanh ghi nguồn/đích và tạo toán hạng tức thời. Register file của RV32I có 32 thanh ghi nguyên 32 bit; x0 luôn đọc bằng 0. RV32E là biến thể giảm còn 16 thanh ghi, nên không nên gán đặc điểm 32 thanh ghi cho mọi cấu hình.

Tầng EX (Execute) dùng ALU thực hiện số học, logic, dịch bit, so sánh nhánh hoặc tính địa chỉ hiệu dụng. Bộ nhân/chia, FPU hay bộ tăng tốc chỉ xuất hiện nếu lõi được thiết kế hỗ trợ chúng. Tầng MEM thực hiện load/store qua bộ nhớ dữ liệu hoặc D-cache; không phải mọi lệnh đều truy cập bộ nhớ. Tầng WB chọn kết quả ALU hoặc dữ liệu load để ghi về thanh ghi đích; store và branch không ghi kết quả như một lệnh cộng thông thường.

Khởi điều khiển phát tín hiệu cho datapath; hazard detection, forwarding và stall xử lý phụ thuộc giữa các lệnh. Các khối CSR, ngắt và ngoại lệ hỗ trợ quản lý trạng thái hệ thống tùy cấu hình. Khi tạo SoC, lõi phải kết nối thêm ROM/Flash, SRAM, timer, GPIO, UART, SPI/I²C và các bộ tăng tốc theo nhu cầu. ISA mở không đồng nghĩa bản thân lõi CPU đã bao gồm toàn bộ những ngoại vi này.

Giải thích pipeline và nhận định

Pipeline là kỹ thuật chia quá trình thực hiện một lệnh thành nhiều tầng, ngăn cách bằng các thanh ghi pipeline. Trong cùng một chu kỳ, các tầng xử lý các lệnh khác nhau: chẳng hạn lệnh I1 ở EX, I2 ở ID và I3 ở IF. Nhờ chồng lấp công việc, CPU không phải chờ I1 hoàn thành toàn bộ mới bắt đầu I2.

Với mô hình năm tầng IF–ID–EX–MEM–WB, nếu không có stall, sau khi nạp đầy pipeline, lý tưởng mỗi chu kỳ hoàn thành một lệnh (CPI xấp xỉ 1 đối với lõi single-issue). Điều này làm tăng thông lượng, tức số lệnh hoàn thành trong một đơn vị thời gian. Tuy nhiên, một lệnh riêng vẫn phải đi qua các tầng cần thiết; pipeline không có nghĩa mỗi lệnh chỉ mất một chu kỳ từ đầu đến cuối.

Ví dụ với bốn lệnh độc lập, trình tự lý tưởng như sau; dấu “–” chỉ thời điểm lệnh chưa bắt đầu hoặc đã hoàn thành:

Lệnh	CK1	CK2	CK3	CK4	CK5	CK6	CK7	CK8
I1	IF	ID	EX	MEM	WB	–	–	–
I2	–	IF	ID	EX	MEM	WB	–	–
I3	–	–	IF	ID	EX	MEM	WB	–
I4	–	–	–	IF	ID	EX	MEM	WB

Với k tầng và n lệnh, pipeline lý tưởng cần $k + n - 1$ chu kỳ. Nếu thực hiện tuần tự từng lệnh qua k bước bằng nhau, cần $k \times n$ chu kỳ với cùng độ dài mỗi bước. Trong ví dụ $k = 5$ và $n = 4$, số chu kỳ giảm từ 20 xuống 8; hệ số tăng tốc là $20/8 = 2,5$. Khi n lớn, hệ số này tiến gần 5 chỉ trong mô hình lý tưởng, các tầng cân bằng và bỏ qua chi phí thanh ghi.

Nhận định cần phân biệt thông lượng và độ trễ. Chu kỳ clock của pipeline phụ thuộc tầng chậm nhất cộng overhead của thanh ghi pipeline; nó thường ngắn hơn chu kỳ của mạch đơn chu kỳ phải đi qua toàn bộ đường xử lý. Trong khi đó, độ trễ một lệnh xấp xỉ k lần chu kỳ pipeline và có thể tăng do overhead hoặc mất cân bằng giữa các tầng. Vì vậy, tần số cao hơn và thông lượng cao hơn không đồng nghĩa độ trễ của từng lệnh luôn giảm.

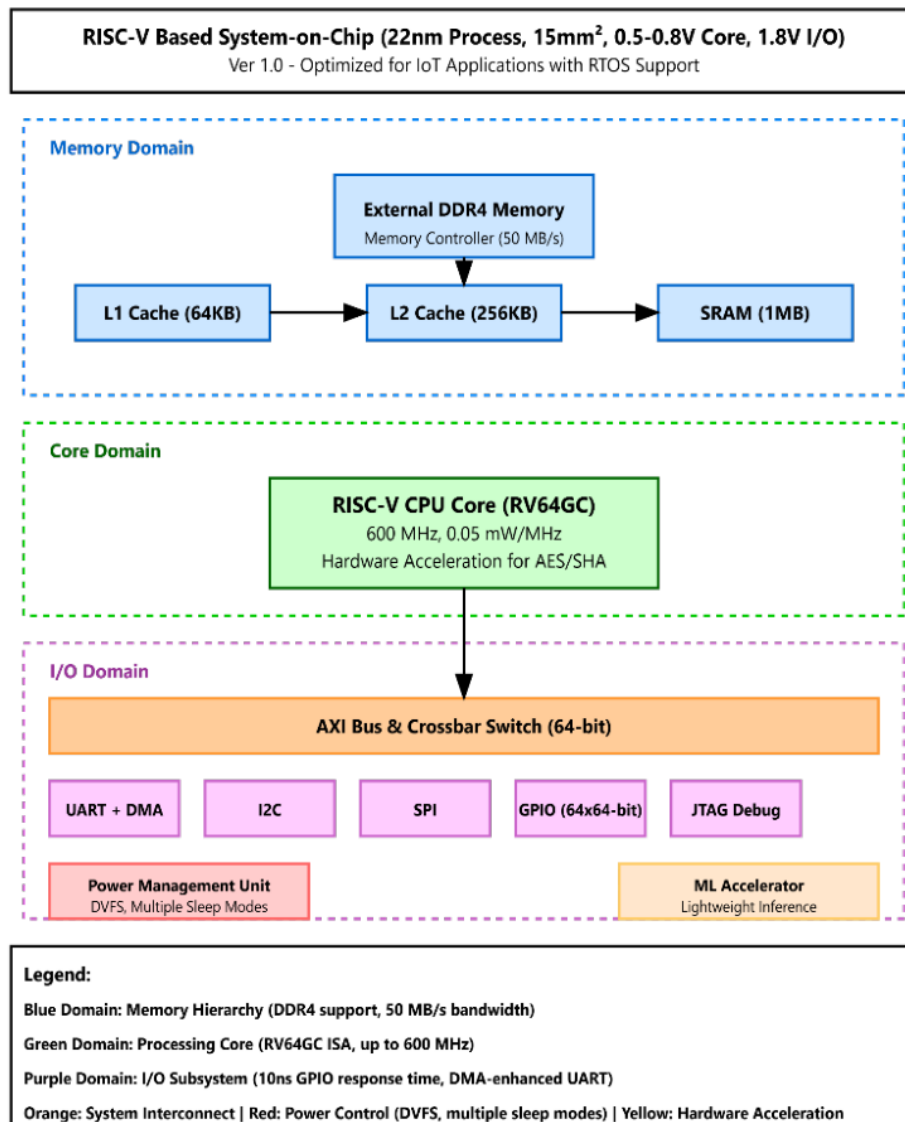
Ví dụ về thời gian: năm khâu đều mất 2 ns thì mạch đơn chu kỳ lý tưởng cần 10 ns/lệnh. Nếu pipeline thêm 0,2 ns overhead mỗi tầng, chu kỳ là 2,2 ns và độ trễ một lệnh khoảng 11 ns; sau khi pipeline đầy, có thể hoàn thành một lệnh mỗi 2,2 ns. Như vậy, độ trễ tăng từ 10 lên 11 ns nhưng thông lượng lý tưởng tăng khoảng $10/2,2 = 4,55$ lần. Đây là ví dụ tính toán, không phải thông số đo trong tài liệu.

Thực tế, phụ thuộc dữ liệu (data hazard), tranh chấp tài nguyên (structural hazard), nhánh/nhảy (control hazard) và cache miss có thể tạo stall hoặc flush. Forwarding, interlock, tách bộ nhớ lệnh/dữ liệu và dự đoán nhánh giúp giảm tổn thất, nhưng không loại bỏ mọi tình huống chờ. Mức tối ưu phải cân bằng hiệu năng, công suất và diện tích.

Các kỹ thuật tối ưu và đánh đổi

Nhóm tối ưu	Cách áp dụng và đánh đổi
Tập lệnh và cấu hình	Chọn RV32I/E và các extension cần thiết. M hỗ trợ nhân/chia; C giảm kích thước mã; F/D hỗ trợ dấu phẩy động. Chỉ thêm chức năng có ích để tránh tăng diện tích và công suất không cần thiết.
Pipeline và hazard	Chia đường tổ hợp để tăng clock; forwarding giảm stall do dữ liệu; dự đoán nhánh giảm thời gian chờ. Pipeline sâu làm tăng chi phí flush và số thanh ghi.
Bộ nhớ và dữ liệu	Cache giảm thời gian truy cập trung bình; TCM giúp thời gian truy cập để dự đoán. DMA, local buffer và tổ chức dữ liệu hợp lý giảm tải CPU và lưu lượng bộ nhớ.
Song song và tăng tốc	Dùng nhiều lõi, vector/SIMD khi được hỗ trợ, lệnh tùy biến hoặc accelerator cho MAC, DSP và mật mã. Cần cân nhắc phần mềm, diện tích và chi phí truyền dữ liệu.
Công suất	Clock gating giảm chuyển mạch; sleep và power gating giảm tiêu thụ lúc nhàn rỗi; DVFS nếu hệ thống hỗ trợ. Phải tính thời gian đánh thức và ràng buộc thời gian thực.
Phần mềm và mô hình ML	Tối ưu compiler, bố trí bộ nhớ, fixed-point/quantization và kernel chuyên biệt. Đánh giá cả độ chính xác lẫn tốc độ, năng lượng và dung lượng bộ nhớ.

Không có phương án tối ưu chung cho mọi ứng dụng. Nút cảm biến cần năng lượng thấp và bộ nhớ nhỏ thường phù hợp lõi đơn giản; tác vụ học máy có nhiều phép MAC có thể hưởng lợi từ CFU hoặc accelerator. Với hệ thống thời gian thực, tính dự đoán được đôi khi quan trọng hơn hiệu năng trung bình.



Hình 5. SoC RISC-V

Trong Hình 5, lõi RISC-V xử lý chương trình; hệ thống cache, SRAM và giao tiếp DRAM cung cấp bộ nhớ; bus AXI/crossbar nối CPU với UART/DMA, I²C, SPI, GPIO và debug. Bộ quản lý nguồn điều phối năng lượng, còn khối tăng tốc ML phục vụ suy luận tại biên. Đây là kiến trúc đề xuất của tác giả; các con số công nghệ, tần số và bộ nhớ ghi trên hình không được hiểu là thông số chung của RISC-V hoặc bằng chứng chip đã được đo kiểm.

Ứng dụng IoT và hệ nhúng

Một nút IoT dùng CPU RISC-V đọc cảm biến qua GPIO, UART, SPI hoặc I²C, thực hiện lọc/điều khiển rồi giao tiếp với mô-đun mạng. Lõi nhỏ, chế độ ngủ và bộ nhớ vừa đủ giúp giảm năng lượng, nhưng khả năng mạng không tự có chỉ vì dùng ISA RISC-V. Khối radio hoặc giao tiếp mạng phải được tích hợp hoặc kết nối thêm.

Khảo sát mô tả PicoRV32 có tính cấu hình và nhiều lựa chọn giao tiếp, phù hợp hệ nhúng nhỏ; MicroRV dùng cho giáo dục/nghiên cứu và có khả năng chạy FreeRTOS; RavenSoC tích hợp thêm ngoại vi và khối analog. Từ đó, lựa chọn lõi phải gắn với nhu cầu cảm biến, hệ điều hành và tài nguyên, thay vì chỉ so sánh tần số clock.

Tiếp tục phân tích yêu cầu SoC IoT như ngoại vi, bảo mật, quản lý năng lượng và khả năng mở rộng. Các cấu hình được tác giả đề xuất cần được phân biệt với kết quả của chip đã triển khai; không dùng chúng như bằng chứng silicon đã được kiểm chứng.

Ứng dụng xử lý tín hiệu và học máy tại biên

Trong xử lý âm thanh, ảnh hoặc dữ liệu cảm biến, lõi RISC-V đảm nhiệm điều khiển và tổ chức dữ liệu; các phép tính lặp như MAC, tích chập hoặc lọc được đưa vào đơn vị chuyên dụng. Cách tổ chức này có thể giảm số lệnh và năng lượng mỗi tác vụ, nhưng cần kiểm tra xem dữ liệu có được cấp đủ nhanh cho bộ tăng tốc hay không.

Khảo sát, mục 4.2, mô tả CFU Playground: ghép lõi mềm RISC-V với Custom Function Unit để tăng tốc những phép toán thường gặp trong ML. CFU nhận toán hạng và trả kết quả theo giao tiếp với CPU; không nên mặc định CFU tự đọc/ghi bộ nhớ. Đây là ví dụ rõ của tối ưu đồng thời phần mềm, ISA và phần cứng.

Khảo sát cũng trình bày nền tảng K210 cùng bộ tăng tốc KPU và ví dụ nhận diện khuôn mặt. Trong hệ thống như vậy, CPU RISC-V quản lý ứng dụng, còn khối chuyên dụng đảm nhiệm phần tính toán mạng nơ-ron. Không thể quy toàn bộ hiệu năng AI cho riêng lõi CPU hay riêng ISA.

Ứng dụng nghiên cứu và phát triển SoC

Lõi RISC-V mã nguồn mở thuận lợi cho học pipeline, kiểm chứng RTL, thử lệnh tùy biến và tạo mẫu trên FPGA. Khả năng thay đổi vi kiến trúc giúp khảo sát PPA hoặc xây dựng SoC chuyên dụng. Tuy vậy, lõi RISC-V thương mại có thể đóng mã nguồn; giấy phép triển khai và các IP đi kèm phải được kiểm tra riêng.

Hình nhóm

